Document number: P0031R0
Project: Programming Language C++, Library Evolution Working Group

Antony Polukhin <antoshkka@gmail.com>

Date: 2015-09-09

A Proposal to Add Constexpr Modifiers to reverse_iterator, move_iterator, array and Range Access

I. Introduction and Motivation

The Standard Library provides a great collention of containers and algorithms, many of which currently lack constexpr support. Even a simple `constexpr` usage requires reimplementing a big bunch of the Standard Library. Consider the simple example:

```
#include <array>
#include <algorithm>

int main() {
        // OK
        constexpr std::array<char, 6> a { 'H', 'e', 'l', 'l', 'o' };

        // Failures:
        // * std::find is not constexpr
        // * std::array::rbegin(), std::array::rend() are not constexpr
        // * std::array::reverse_iterator is not constexpr
        constexpr auto it = std::find(a.rbegin(), a.rend(), 'H');
}
```

This document proposes a number of `constexpr` additions, to the Standard Library, that are easy to implement and do not require additional compiler support. A proof of concept implementation, tested with gcc 5.2 and clang 3.6,2 is available at: https://github.com/apolukhin/constexpr_additions/blob/master/constexpr_additions.cpp.

This proposal concentrates on simple containers and iterators, deferring `constexpr` algorithms to a separate proposal.

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to existing headers `<iterator>` and `<array>` such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language, and it has been implemented in standard C++. The proposal depends on the proposed resolution for LWG2296. LWG2296 in Revision D94 of the Library Active Issues List.

III. Design Decisions

A. `std::array` must behave as a native array

`std::array` was designed to be an extremely optimized wrapper around array that has the same layout and constraints as a native array. At this moment this is satisfied only partially, because arrays are usable in `constexpr` functions, while `std::array` is not.

B. `std::reverse_iterator` must only reverse

`std::reverse_iterator` must not add additional constraints on an underlying iterator. It must be as close as possible to the underlying `base()` iterator, so if `base()` iterator is a pointer or iterator that could be used in `constexpr` expressions, then `std::reverse_iterator` must be thusly usable too.

C. `std::move_iterator` - just move on dereference

Just like `std::reverse_iterator`, `std::move_iterator` must not add additional constraints on an underlying iterator. If base() iterator could be used in `constexpr` expressions, then `std::move_iterator` must be thusly usable too.

D. Range access functions and iterators related functions constexprness must be added

If some of the one-line range access functions have no `constexpr`, then just add it.

E. `std::array` comparisons and `swap`/`fill` functions are not affected by this proposal

Currently comparisons and `swap`/`fill` may be implemented with the help of algorithms from <algorithm> header. Marking comparisons with `constexpr` will break that ability and will potentially lead to performance degradations.

IV. Proposed modifications for N4527

All the additions to the Standard are marked with underlined green.

A. Modifications to "24.3 Header <iterator> synopsis" [iterator.synopsis]

```
// 24.4.4, iterator operations:
template <class InputIterator, class Distance>
constexpr void advance(InputIterator& i, Distance n);

template <class InputIterator>
constexpr typename iterator_traits<InputIterator>::difference_type
distance(InputIterator first, InputIterator last);

template <class ForwardIterator>
constexpr ForwardIterator next(ForwardIterator x,
typename std::iterator_traits<ForwardIterator>::difference_type n = 1);

template <class BidirectionalIterator>
constexpr BidirectionalIterator prev(BidirectionalIterator x,
typename std::iterator_traits<BidirectionalIterator>::difference_type n = 1);

// 24.5, predefined iterators:
template <class Iterator> class reverse_iterator;

template <class Iterator1, class Iterator2>
constexpr bool operator==(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator!=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr auto operator-(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
template <class Iterator>
constexpr reverse_iterator<Iterator>
        operator+(
        typename reverse_iterator<Iterator>::difference_type n,
        const reverse_iterator<Iterator>& x);

template <class Iterator>
constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i);


template <class Iterator> class move_iterator;

template <class Iterator1, class Iterator2>
constexpr bool operator==(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

```
template <class Iterator1, class Iterator2>
constexpr bool operator!=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);

template <class Iterator1, class Iterator2>
constexpr auto operator-(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
template <class Iterator>
constexpr move_iterator<Iterator> operator+(
        typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
template <class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i);

// 24.7, range access:
template <class C> constexpr auto begin(C& c) -> decltype(c.begin());
template <class C> constexpr auto begin(const C& c) -> decltype(c.begin());
template <class C> constexpr auto end(C& c) -> decltype(c.end());
template <class C> constexpr auto end(const C& c) -> decltype(c.end());
template <class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
template <class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
template <class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c))) -> decltype(std::begin(c));
template <class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c))) -> decltype(std::end(c));
template <class C> constexpr auto rbegin(C& c) -> decltype(c.rbegin());
template <class C> constexpr auto rbegin(const C& c) -> decltype(c.rbegin());
template <class C> constexpr auto rend(C& c) -> decltype(c.rend());
template <class C> constexpr auto rend(const C& c) -> decltype(c.rend());
template <class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]);
template <class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]);
template <class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il);
template <class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il);
template <class C> constexpr auto crbegin(const C& c) -> decltype(std::rbegin(c));
template <class C> constexpr auto crend(const C& c) -> decltype(std::rend(c));
```

B. Modifications to "24.4.4 Iterator operations" [iterator.operations]

```
template <class InputIterator, class Distance>
constexpr void advance(InputIterator& i, Distance n);

template <class InputIterator>
constexpr typename iterator_traits<InputIterator>::difference_type
distance(InputIterator first, InputIterator last);

template <class ForwardIterator>
constexpr ForwardIterator next(ForwardIterator x,
        typename std::iterator_traits<ForwardIterator>::difference_type n = 1);

template <class BidirectionalIterator>
constexpr BidirectionalIterator prev(BidirectionalIterator x,
        typename std::iterator_traits<BidirectionalIterator>::difference_type n = 1);
```

C. Modifications to "24.5.1.1 Class template reverse_iterator" [reverse.iterator]

```
namespace std {
template <class Iterator>
class reverse_iterator {
public:
        typedef Iterator iterator_type;
        typedef typename iterator_traits<Iterator>::iterator_category iterator_category;
        typedef typename iterator_traits<Iterator>::value_type value_type;
        typedef typename iterator_traits<Iterator>::difference_type difference_type;
        typedef typename iterator_traits<Iterator>::pointer pointer;
        typedef typename iterator_traits<Iterator>::reference reference;
```

```
        constexpr reverse_iterator();
        constexpr explicit reverse_iterator(Iterator x);
        template <class U> constexpr reverse_iterator(const reverse_iterator<U>& u);
        template <class U> constexpr reverse_iterator& operator=(const reverse_iterator<U>& u);
        constexpr Iterator base() const;         // explicit

        constexpr reference operator*() const;
        constexpr pointer operator->() const;
        constexpr reverse_iterator& operator++();
        constexpr reverse_iterator operator++(int);
        constexpr reverse_iterator& operator--();
        constexpr reverse_iterator operator--(int);

        constexpr reverse_iterator operator+ (difference_type n) const;
        constexpr reverse_iterator& operator+=(difference_type n);
        constexpr reverse_iterator operator- (difference_type n) const;
        constexpr reverse_iterator& operator-=(difference_type n);
        constexpr unspecified operator[](difference_type n) const;
protected:
        Iterator current;
};

template <class Iterator1, class Iterator2>
constexpr bool operator==(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator!=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<=(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr auto operator-(
        const reverse_iterator<Iterator1>& x,
        const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
template <class Iterator>
constexpr reverse_iterator<Iterator>
        operator+(
        typename reverse_iterator<Iterator>::difference_type n,
        const reverse_iterator<Iterator>& x);

template <class Iterator>
constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i);

}
```

D. Modifications to "24.5.1.3 reverse_iterator operations" [reverse.iter.ops]

24.5.1.3.1 reverse_iterator constructor
constexpr reverse_iterator();
Effects: Value initializes current. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type Iterator.

constexpr explicit reverse_iterator(Iterator x);
Effects: Initializes current with x.

template <class U> constexpr reverse_iterator(const reverse_iterator<U> &u);
Effects: Initializes current with u.current.

24.5.1.3.2 reverse_iterator::operator=

```
template <class U>
constexpr  reverse_iterator& operator=(const reverse_iterator<U>& u);
Effects: Assigns u.base() to current.
Returns: *this.

24.5.1.3.3 Conversion

constexpr Iterator base() const;
Returns: current.

24.5.1.3.4 operator*
constexpr reference operator*() const;
Effects: Iterator tmp = current; return *--tmp;

24.5.1.3.5 operator->
constexpr pointer operator->() const;
Returns: std::addressof(operator*()).

24.5.1.3.6 operator++
constexpr reverse_iterator& operator++();
Effects: --current;
Returns: *this.

constexpr reverse_iterator operator++(int);
Effects: reverse_iterator tmp = *this; --current; return tmp;

24.5.1.3.7 operator--

constexpr reverse_iterator& operator--();
Effects: ++current
Returns: *this.

constexpr reverse_iterator operator--(int);
Effects: reverse_iterator tmp = *this; ++current; return tmp;

24.5.1.3.8 operator+

constexpr reverse_iterator
operator+(typename reverse_iterator<Iterator>::difference_type n) const;
Returns: reverse_iterator(current-n).

24.5.1.3.9 operator+=

constexpr reverse_iterator&
operator+=(typename reverse_iterator<Iterator>::difference_type n);
Effects: current -= n;
Returns: *this.

24.5.1.3.10 operator-
constexpr reverse_iterator
    operator-(typename reverse_iterator<Iterator>::difference_type n) const;
Returns: reverse_iterator(current+n).

24.5.1.3.11 operator-=

constexpr reverse_iterator&
    operator-=(typename reverse_iterator<Iterator>::difference_type n);
Effects: current += n;
Returns: *this.

24.5.1.3.12 operator[]

constexpr unspecified operator[](
    typename reverse_iterator<Iterator>::difference_type n) const;
Returns: current[-n-1].

24.5.1.3.13 operator==

template <class Iterator1, class Iterator2>
constexpr bool operator==(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current == y.current.

24.5.1.3.14 operator<

template <class Iterator1, class Iterator2>
constexpr bool operator<(
```

```
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current > y.current.

24.5.1.3.15 operator!=

template <class Iterator1, class Iterator2>
constexpr bool operator!=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current != y.current.

24.5.1.3.16 operator>

template <class Iterator1, class Iterator2>
constexpr bool operator>(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current < y.current.

24.5.1.3.17 operator>=

template <class Iterator1, class Iterator2>
constexpr bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current <= y.current.

24.5.1.3.18 operator<=

template <class Iterator1, class Iterator2>
constexpr bool operator<=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
Returns: x.current >= y.current.

24.5.1.3.19 operator-

template <class Iterator1, class Iterator2>
constexpr auto operator-(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
Returns: y.current - x.current.

24.5.1.3.20 operator+

template <class Iterator>
constexpr reverse_iterator<Iterator> operator+(
    typename reverse_iterator<Iterator>::difference_type n,
    const reverse_iterator<Iterator>& x);
Returns: reverse_iterator<Iterator> (x.current - n).

24.5.1.3.21 Non-member function make_reverse_iterator()
template <class Iterator>
constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i);
Returns: reverse_iterator<Iterator>(i).
```

E. Modifications to "24.5.3.1 Class template move_iterator" [move.iterator]

```
namespace std {

template <class Iterator>
class move_iterator {
public:
        typedef Iterator iterator_type;
        typedef typename iterator_traits<Iterator>::iterator_category iterator_category;
        typedef typename iterator_traits<Iterator>::value_type value_type;
        typedef typename iterator_traits<Iterator>::difference_type difference_type;
        typedef typename iterator_traits<Iterator>::pointer pointer;
        typedef typename iterator_traits<Iterator>::reference reference;

        constexpr move_iterator();
        constexpr explicit move_iterator(Iterator x);
        template <class U> constexpr move_iterator(const move_iterator<U>& u);
        template <class U> constexpr move_iterator& operator=(const move_iterator<U>& u);

        constexpr Iterator base() const;         // explicit
```

```
        constexpr reference operator*() const;
        constexpr pointer operator->() const;

        constexpr move_iterator& operator++();
        constexpr move_iterator operator++(int);
        constexpr move_iterator& operator--();
        constexpr move_iterator operator--(int);

        constexpr move_iterator operator+ (difference_type n) const;
        constexpr move_iterator& operator+=(difference_type n);
        constexpr move_iterator operator- (difference_type n) const;
        constexpr move_iterator& operator-=(difference_type n);
        constexpr unspecified operator[](difference_type n) const;
protected:
        Iterator current;
};


template <class Iterator1, class Iterator2>
constexpr bool operator==(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator!=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator<=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
constexpr bool operator>=(
        const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);

template <class Iterator1, class Iterator2>
constexpr auto operator-(
        const move_iterator<Iterator1>& x,
        const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
template <class Iterator>
constexpr move_iterator<Iterator> operator+(
        typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
template <class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i);

}
```

F. Modifications to "24.5.3.3 move_iterator operations" [move.iter.ops]

24.5.3.3.1 move_iterator constructors

constexpr move_iterator();
Effects: Constructs a move_iterator, value initializing current. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type Iterator.

constexpr explicit move_iterator(Iterator i);
Effects: Constructs a move_iterator, initializing current with i.

template <class U> constexpr move_iterator(const move_iterator<U>& u);
Effects: Constructs a move_iterator, initializing current with u.base().
Requires: U shall be convertible to Iterator.

24.5.3.3.2 move_iterator::operator=

template <class U> constexpr move_iterator& operator=(const move_iterator<U>& u);
Effects: Assigns u.base() to current.
Requires: U shall be convertible to Iterator.

24.5.3.3.3 move_iterator conversion

constexpr Iterator base() const;
Returns: current.

24.5.3.3.4 move_iterator::operator*

```
constexpr reference operator*() const;
Returns: static_cast<reference>(*current).

24.5.3.3.5 move_iterator::operator->

constexpr pointer operator->() const;
Returns: current.

24.5.3.3.6 move_iterator::operator++

constexpr move_iterator& operator++();
Effects: ++current.
Returns: *this.

constexpr move_iterator operator++(int);
Effects: move_iterator tmp = *this; ++current; return tmp;

24.5.3.3.7 move_iterator::operator--

constexpr move_iterator& operator--();
Effects: --current.
Returns: *this.

constexpr move_iterator operator--(int);
Effects: move_iterator tmp = *this; --current; return tmp;

24.5.3.3.8 move_iterator::operator+

constexpr move_iterator operator+(difference_type n) const;
Returns: move_iterator(current + n).

24.5.3.3.9 move_iterator::operator+=

constexpr move_iterator& operator+=(difference_type n);
Effects: current += n.
Returns: *this.

24.5.3.3.10 move_iterator::operator-

constexpr move_iterator operator-(difference_type n) const;
Returns: move_iterator(current - n).

24.5.3.3.11 move_iterator::operator-=

constexpr move_iterator& operator-=(difference_type n);
Effects: current -= n.
Returns: *this.

24.5.3.3.12 move_iterator::operator[]

constexpr unspecified operator[](difference_type n) const;
Returns: std::move(current[n]).

24.5.3.3.13 move_iterator comparisons

template <class Iterator1, class Iterator2>
constexpr bool operator==(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: x.base() == y.base().

template <class Iterator1, class Iterator2>
constexpr bool operator!=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: !(x == y).

template <class Iterator1, class Iterator2>
constexpr bool operator<(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: x.base() < y.base().

template <class Iterator1, class Iterator2>
constexpr bool operator<=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: !(y < x).

template <class Iterator1, class Iterator2>
constexpr bool operator>(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: y < x.

template <class Iterator1, class Iterator2>
constexpr bool operator>=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
Returns: !(x < y).
```

```
24.5.3.3.14 move_iterator non-member functions

template <class Iterator1, class Iterator2>
constexpr auto operator-(
    const move_iterator<Iterator1>& x,
    const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
Returns: x.base() - y.base().

template <class Iterator>
constexpr move_iterator<Iterator> operator+(
    typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
Returns: x + n.

template <class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i);
Returns: move_iterator<Iterator>(i).
```

G. Modifications to "24.7 Range access" [iterator.range]

```
In addition to being available via inclusion of the <iterator> header, the function templates in 24.7 are
available when any of the following headers are included: <array>, <deque>, <forward_list>, <list>,
<map>, <regex>, <set>, <string>, <unordered_map>, <unordered_set>, and <vector>.

template <class C> constexpr auto begin(C& c) -> decltype(c.begin());
template <class C> constexpr auto begin(const C& c) -> decltype(c.begin());
Returns: c.begin().

template <class C> constexpr auto end(C& c) -> decltype(c.end());
template <class C> constexpr auto end(const C& c) -> decltype(c.end());
Returns: c.end().

template <class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
Returns: array.

template <class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
Returns: array + N.

template <class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
    -> decltype(std::begin(c));
Returns: std::begin(c).

template <class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
    -> decltype(std::end(c));
Returns: std::end(c).

template <class C> constexpr auto rbegin(C& c) -> decltype(c.rbegin());
template <class C> constexpr auto rbegin(const C& c) -> decltype(c.rbegin());
Returns: c.rbegin().

template <class C> constexpr auto rend(C& c) -> decltype(c.rend());
template <class C> constexpr auto rend(const C& c) -> decltype(c.rend());
Returns: c.rend().

template <class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]);
Returns: reverse_iterator<T*>(array + N).

template <class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]);
Returns: reverse_iterator<T*>(array).

template <class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il);
Returns: reverse_iterator<const E*>(il.end()).

template <class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il);
Returns: reverse_iterator<const E*>(il.begin()).

template <class C> constexpr auto crbegin(const C& c) -> decltype(std::rbegin(c));
Returns: std::rbegin(c).

template <class C> constexpr auto crend(const C& c) -> decltype(std::rend(c));
Returns: std::rend(c).
```

H. Modifications to "23.3.2.1 Class template array overview" [array.overview]

We propose no changes to `std::array`'s comparison, swap, and fill functions.

```
namespace std {

template <class T, size_t N>
struct array {
        // types:
        typedef T& reference;
        typedef const T& const_reference;
        typedef implementation-defined iterator;
        typedef implementation-defined const_iterator;
        typedef size_t size_type;
        typedef ptrdiff_t difference_type;
        typedef T value_type;
        typedef T* pointer;
        typedef const T* const_pointer;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

        T elems[N]; // exposition only

        // no explicit construct/copy/destroy for aggregate type

        void fill(const T& u);
        void swap(array&) noexcept(noexcept(swap(declval<T&>(), declval<T&>())));

        // iterators:
        constexpr iterator begin() noexcept;
        constexpr const_iterator begin() const noexcept;
        constexpr iterator end() noexcept;
        constexpr const_iterator end() const noexcept;

        constexpr reverse_iterator rbegin() noexcept;
        constexpr const_reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator rend() noexcept;
        constexpr const_reverse_iterator rend() const noexcept;

        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;
        constexpr const_reverse_iterator crbegin() const noexcept;
        constexpr const_reverse_iterator crend() const noexcept;

        // capacity:
        constexpr bool empty() const noexcept;
        constexpr size_type size() const noexcept;
        constexpr size_type max_size() const noexcept;
        // element
        constexpr reference operator[](size_type n);
        constexpr const_reference operator[](size_type n) const;
        constexpr reference at(size_type n);
        constexpr const_reference at(size_type n) const;
        constexpr reference front();
        constexpr const_reference front() const;
        constexpr reference back();
        constexpr const_reference back() const;

        constexpr T * data() noexcept;
        constexpr const T * data() const noexcept;
};

}
```

I. Modifications to "23.3.2.5 array::data" [array.data]

```
23.3.2.5 array::data

constexpr T* data() noexcept;
constexpr const T* data() const noexcept;
Returns: elems.
```

J. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_array_constexpr`.

V. Revision History

Revision 1:

- Initial proposal

VI. References

[Boost15] Boost array documentation. Available online at http://www.boost.org/doc/libs/1_58_0/doc/html/array.html

[Boost15a] Boost array addition to support `constexpr`. Available online at https://github.com/boostorg/array/pull/4/files

[N3690] Working Draft, Standard for Programming Language C ++. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4527.pdf

[N4527] "Draft N3690 : Why is reference std::array::operator[] not constexpr ?" discussion. Available online at https://groups.google.com/a/isocpp.org/forum/#!topic/std-discussion/nrAu_YbCbYM

[Antony Polukhin] "[std-proposals] Towards a constexpr usable Standard Library" discussion. Available online at https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/wdh1lLlRnX4

[Antony Polukhin] Proof of concept implementation. Available online at https://github.com/apolukhin/constexpr_additions/blob/master/constexpr_additions.cpp

[rhalbersma] std::array only extended proof of concept. Available online at https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/array.hpp

VII. Acknowledgements

Walter E. Brown provided numerous comments, corrections, and suggestions for this proposal.